

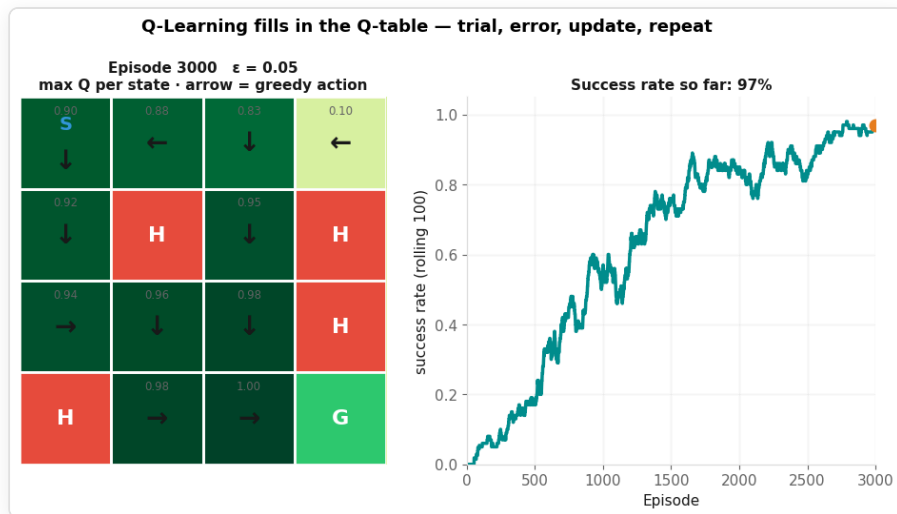
Q-Learning

Applied Machine Learning — Session 4, Chapter 2

From Ch10 to Ch11

Ch10: if we *know* $Q(s, a)$, the policy is trivial: argmax . **Ch11:** how do we **learn** Q from experience — no map, no transition probabilities, no reward function?

Answer: start with $Q = 0$, act ϵ -greedy, and after *every step* nudge $Q(s, a)$ toward what we just observed.



The Q-Table

Q-table (16 states $\times$ 4 actions) – all zeros at the start

	←	↓	→	↑
state 0:	0.00	0.00	0.00	0.00
state 1:	0.00	0.00	0.00	0.00
...				
state 14:	0.00	0.00	0.00	0.00
state 15:	0.00	0.00	0.00	0.00

← goal (never left)

- One number per (state, action): *"how good is it to do a in s?"*
- **Policy** = `argmax` per row
- **Learning** = updating these numbers from experience

The TD Update (1/2): the idea

After one step $(s, a) \rightarrow r, s'$ we have new information:

```
what we thought:  Q(s, a)
what we just saw:  r + γ · max_{a'} Q(s', a')      ← "target"
                   ^         ^
                   reward now best we believe we can still get from s'
```

Move the estimate a little toward the target:

```
Q(s, a) ← Q(s, a) + α · ( target - Q(s, a) )
                        |—— TD error ——|
```

α = learning rate (how big a step). If the episode ended at s' , there is no future: target = r.

The TD Update (2/2): a worked step

GridWorld, $\alpha = 0.1$, $\gamma = 0.99$, Q all zeros. The agent stands in state 14 and steps $\rightarrow$ into the goal:

```
target  = 1.0 + (episode ended  $\rightarrow$  no future) = 1.0
TD err  = 1.0 - 0.0 = 1.0
 $Q(14, \rightarrow) = 0.0 + 0.1 \cdot 1.0 = 0.10$ 
```

Next episode: from state 13 it steps $\rightarrow$ into 14 (reward -0.01):

```
target  =  $-0.01 + 0.99 \cdot \max Q(14, \cdot) = -0.01 + 0.99 \cdot 0.10 = 0.089$ 
 $Q(13, \rightarrow) = 0.0 + 0.1 \cdot (0.089 - 0.0) = 0.0089$ 
```

$\rightarrow$ Value **flows backwards** from the goal, one cell per successful visit. Small numbers, but the *ordering* of actions is what matters for `argmax`.

The Full Algorithm

```
Q, epsilon = np.zeros((n_states, n_actions)), 1.0
for episode in range(n_episodes):
    state = env_reset()
    for step in range(max_steps):
        #  $\epsilon$ -greedy
        if rng.random() < epsilon: action = rng.integers(n_actions)    # explore
        else: action = np.argmax(Q[state])    # exploit

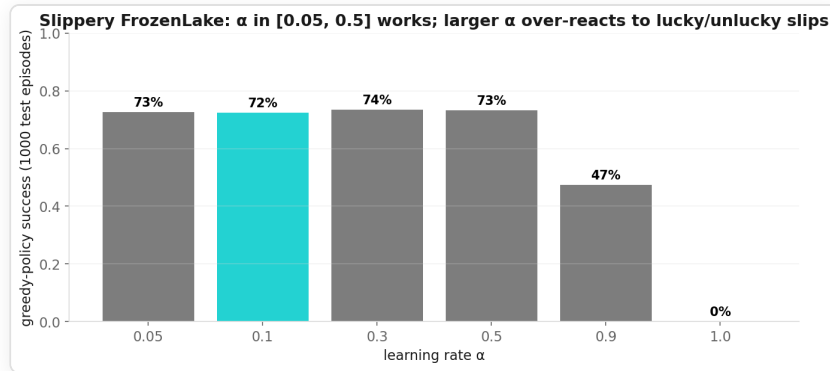
        next_state, reward, done = env_step(state, action)    # gymnasium: 5-tuple, see demo
        target = reward + (0 if done else gamma * np.max(Q[next_state]))    # TD target
        Q[state, action] += alpha * (target - Q[state, action])    # TD update

        state = next_state
        if done: break
    epsilon = max(epsilon * 0.999, 0.01)    # decay
```

Hyperparameters (the ones we use)

Parameter	Symbol	Ours	Typical
learning rate	α	0.1	0.05 – 0.5
discount	γ	0.99	0.9 – 0.99
exploration	ϵ	$1 \rightarrow 0.01$	$\times 0.999$ / episode
episodes		3 000 – 5 000	$10^3 - 10^5$

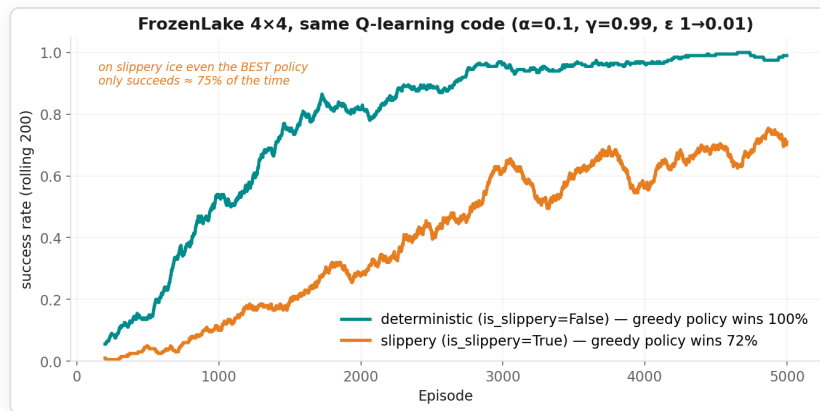
- α : step size of each update
- γ : how far value flows backwards
- more episodes = better, slower



FrozenLake (Gymnasium) — same map, two difficulty levels

SFFF	S start (0)
FHFH	F frozen
FFFH	H hole → episode ends, reward 0
HFFG	G goal (15) → reward +1

- 16 states, 4 actions ($\leftarrow 0 \downarrow 1 \rightarrow 2 \uparrow 3$)
- reward **only** at the goal → sparse signal
- `is_slippery=True`: intended move with $p = 1/3$, else sideways



Same code, two worlds: deterministic → $\sim 100\%$; slippery → $\sim 70\%$. On slippery ice even the *optimal* policy only wins ≈ 3 of 4 episodes.

Why is RL hard? (and why does it still work?)

Sparse rewards — FrozenLake: one +1 at the very end, zeros everywhere else. Random exploration must stumble into the goal first.

Stochasticity — same state, same action, different outcome. Estimates need many samples ($\rightarrow$ small α).

Credit assignment — which of the 20 earlier moves caused the success? Answer: γ propagates value backwards, one step per success.

It still works because: tabular Q-learning provably converges to Q^* if every (s, a) is visited infinitely often and α decreases suitably. In practice: enough episodes + ϵ -decay.

Quick Check

1. $\alpha = 1$ in a deterministic world. What happens? And on slippery ice?
2. $\gamma = 0$. Which Q-values ever become non-zero in FrozenLake (reward only at the goal)?
3. After training, ϵ is still 0.05. Is the "success rate during training" the success rate of the *learned policy*?

Quick Check

1. $\alpha = 1$ in a deterministic world. What happens? And on slippery ice?
2. $\gamma = 0$. Which Q-values ever become non-zero in FrozenLake (reward only at the goal)?
3. After training, ϵ is still 0.05. Is the "success rate during training" the success rate of the *learned policy*?
 - 1. Deterministic: fine — the target is exact, so overwrite it. Slippery: each slip overwrites the estimate; Q jumps around and never settles (see the α bar chart: 0 % at $\alpha = 1$).
 - 2. Only entries whose step lands *in* the goal — essentially $Q(14, \rightarrow)$ (on slippery ice also slips from 14). Nothing flows backwards $\rightarrow$ the agent cannot find the goal from the start.
 - 3. No — 5 % of the actions are random. Evaluate the greedy policy separately (`evaluate_greedy` in the demo).

Now: Live Demo (~8 min)

→ `02-examples/ch11_rl_algorithms_examples.ipynb`

1. Wrap Gymnasium's FrozenLake in our `reset / step` interface
2. Q-Learning in ~25 lines — train on solid ice
3. **Same code** on slippery ice → compare success rates
4. Q-table heatmaps + greedy policy arrows: read what the agent learned

Now: Exercise (~10 min)

→ `03-exercises/ch11_rl_algorithms_exercises.ipynb`

Tasks 1–4 (10 min): Q-table → ϵ -greedy → TD update → fill three lines in the given training loop. Each task has a ► check cell with the expected numbers.

Bonus A: run *your* loop on slippery FrozenLake (Gymnasium) — why not 100 %? **Bonus B:** SARSA — change one line, compare learning curves.

Further Reading (not needed for the capstone)

Idea	One line	Where
SARSA (on-policy)	target uses the action you <i>actually</i> take next: $r + \gamma Q(s', a')$ — learns the value of the ϵ -greedy behaviour → more cautious near cliffs	Bonus B
Deep Q-Network (DQN)	replace the table by a neural network $Q(s, a; \theta)$ — Atari from pixels (2013)	
Policy gradient / PPO	learn $\pi(a s)$ directly; needed for continuous actions (robot joints); PPO is the workhorse behind RLHF	
Model-based RL	learn P and R , then plan (AlphaZero uses a known model + search)	
Off- vs on-policy	Q-learning learns the <i>greedy</i> policy while behaving ϵ -greedy (off-policy); SARSA learns the policy it follows (on-policy)	

Key Takeaways

- Q-Learning: **Q-table** + **ϵ -greedy** + **TD update**, repeated over many episodes
- Target $r + \gamma \cdot \max Q(s', \cdot)$; TD error = target – estimate; move by α
- Value flows **backwards** from the reward — that solves credit assignment
- Same code works on deterministic and stochastic worlds; the **environment** sets the ceiling
- Evaluate the **greedy** policy, not the training run (train $\neq$ test — again)

Next: Chapter 12

Capstone: End-to-End ML Workflow

"Time to put everything together."